

Relatório Técnico - Simulação de Controle Semafórico com Listas Encadeadas em Java

João Henrique Pires Rabelo Ramos
Engenharia de Software - ICEV
joaohenrique.ramos@somosicev.com

Romerson Claudino Gonçalves
Engenharia de Software - ICEV
romerson.filho@somosicev.com

Resumo—Este relatório apresenta o desenvolvimento de uma lista circular personalizada em Java, projetada para simular as transições de estados de um sistema semafórico. A motivação advém das limitações de estruturas estáticas ou baseadas em arrays na modelagem de processos cíclicos, comuns no contexto do tráfego urbano.

A estrutura desenvolvida utiliza nós que encapsulam estados e durações, possibilitando transições suaves, customizáveis e eficientes em tempo constante. Os testes práticos demonstraram baixa utilização de memória, alta modularidade do código e viabilidade para aplicações embarcadas e educacionais.

I. INTRODUÇÃO

O crescimento das áreas urbanas intensificou a complexidade do tráfego, exigindo sistemas de controle cada vez mais eficientes e adaptáveis. Simulações computacionais surgem como alternativa viável à implementação direta, mitigando riscos e custos associados a testes em campo [1], [2].

Diversas ferramentas, como Arena, Vissim e SUMO, têm sido aplicadas com sucesso na modelagem de cenários urbanos, permitindo a avaliação de impactos e intervenções antes da adoção real [3], [4]. Essas simulações, muitas vezes integradas a dados em tempo real e algoritmos de IA, proporcionam soluções mais eficazes no planejamento do tráfego [5].

Neste contexto, este trabalho explora o uso de estruturas de dados dinâmicas, com ênfase em listas circulares implementadas em Java, para representar ciclos de controle semafórico, contribuindo para o desenvolvimento de sistemas inteligentes de transporte.

II. REFERENCIAL TEÓRICO

Listas encadeadas são estruturas fundamentais em ciência da computação, possibilitando manipulações dinâmicas com baixo custo computacional [6]. Sua variante circular, onde o último nó aponta para o primeiro, é ideal para modelar sistemas cíclicos, como sinais de trânsito [7].

Com a flexibilidade da linguagem Java e sua abordagem orientada a objetos, é possível criar classes específicas que representam nós com atributos de estado e tempo, permitindo a criação de ciclos contínuos com transições controladas [8].

Além disso, listas circulares eliminam a necessidade de condicionais para reinício do ciclo, tornando o código mais limpo e eficiente. Aplicações modernas incluem integração com sensores e dispositivos embarcados, e até com algoritmos de aprendizado por reforço em sistemas inteligentes [9].

III. APLICAÇÃO PRÁTICA

Na implementação do sistema semafórico, utilizamos:

- **Lista circular:** armazena os estados do semáforo (verde, amarelo, vermelho) como Strings.
- **Lista encadeada:** utilizada para modelar interseções e ruas, formando a estrutura do grafo.
- **Pilha encadeada:** usada para reconstrução do caminho gerado pelo algoritmo de Dijkstra.
- **Nós simples e duplos:** usados nas listas acima conforme a necessidade de iteração ou retrocesso.

Foram implementados três modos de semáforo:

- 1) **Ciclo Fixo:** transições com tempo estático.
- 2) **Tempo de Espera:** adapta o tempo de sinal conforme a quantidade de carros.
- 3) **Consumo:** alterna entre os dois modos anteriores, ativando tempo de espera em horários de pico.

A. Modelagem do Sistema

O sistema é composto por duas entidades principais: **ruas** e **interseções**. Cada *rua* é capaz de calcular o **tempo de travessia**, o **consumo por travessia** e a **capacidade de veículos** com base em seu comprimento. Para isso, foi adotado o comprimento médio de um carro como 4 metros, permitindo estimar a quantidade máxima de veículos suportados proporcionalmente ao tamanho da via.

A classe *Veiculo*, ao atravessar uma sequência de ruas, acumula o tempo de viagem e o consumo de combustível baseado nas características de cada rua. Esses dados são armazenados internamente para análise posterior do percurso completo.

As *interseções* foram modeladas com dois semáforos: um para o fluxo Norte-Sul e outro para Leste-Oeste. Cada semáforo possui um atributo booleano *isVermelho*, indicando se o sinal está fechado. Como os veículos têm acesso às ruas diretamente, o sistema verifica qual direção está sendo tomada para consultar o semáforo correspondente. Se, por exemplo, o veículo entrar em uma rua horizontal, o estado do semáforo Leste-Oeste é avaliado para autorizar ou bloquear o movimento.

A movimentação dos veículos ocorre a partir de caminhos gerados aleatoriamente pela classe *Roteador*, que simula rotas urbanas utilizando algoritmos de busca. Durante a travessia, o veículo avalia o semáforo da próxima interseção e avança apenas quando o sinal está verde, respeitando o ciclo semafórico definido na lista circular.

IV. RESULTADOS E LIMITAÇÕES

[conference]IEEEtran [utf8]inputenc [T1]fontenc [brasil]babel graphicx url hyperref amsmath

Resumo—Este relatório apresenta o desenvolvimento de uma lista circular personalizada em Java, projetada para simular as transições de estados de um sistema semafórico. A motivação advém das limitações de estruturas estáticas ou baseadas em arrays na modelagem de processos cíclicos, comuns no contexto do tráfego urbano.

A estrutura desenvolvida utiliza nós que encapsulam estados e durações, possibilitando transições suaves, customizáveis e eficientes em tempo constante. Os testes práticos demonstraram baixa utilização de memória, alta modularidade do código e viabilidade para aplicações embarcadas e educacionais.

V. INTRODUÇÃO

O crescimento das áreas urbanas intensificou a complexidade do tráfego, exigindo sistemas de controle cada vez mais eficientes e adaptáveis. Simulações computacionais surgem como alternativa viável à implementação direta, mitigando riscos e custos associados a testes em campo [1], [2].

Diversas ferramentas, como Arena, Vissim e SUMO, têm sido aplicadas com sucesso na modelagem de cenários urbanos, permitindo a avaliação de impactos e intervenções antes da adoção real [3], [4]. Essas simulações, muitas vezes integradas a dados em tempo real e algoritmos de IA, proporcionam soluções mais eficazes no planejamento do tráfego [5].

Neste contexto, este trabalho explora o uso de estruturas de dados dinâmicas, com ênfase em listas circulares implementadas em Java, para representar ciclos de controle semafórico, contribuindo para o desenvolvimento de sistemas inteligentes de transporte.

VI. REFERENCIAL TEÓRICO

Listas encadeadas são estruturas fundamentais em ciência da computação, possibilitando manipulações dinâmicas com baixo custo computacional [6]. Sua variante circular, onde o último nó aponta para o primeiro, é ideal para modelar sistemas cíclicos, como sinais de trânsito [7].

Com a flexibilidade da linguagem Java e sua abordagem orientada a objetos, é possível criar classes específicas que representam nós com atributos de estado e tempo, permitindo a criação de ciclos contínuos com transições controladas [8].

Além disso, listas circulares eliminam a necessidade de condicionais para reinício do ciclo, tornando o código mais limpo e eficiente. Aplicações modernas incluem integração com sensores e dispositivos embarcados, e até com algoritmos de aprendizado por reforço em sistemas inteligentes [9].

VII. APLICAÇÃO PRÁTICA

Na implementação do sistema semafórico, utilizamos:

- **Lista circular:** armazena os estados do semáforo (verde, amarelo, vermelho) como Strings.
- **Lista encadeada:** utilizada para modelar interseções e ruas, formando a estrutura do grafo.
- **Pilha encadeada:** usada para reconstrução do caminho gerado pelo algoritmo de Dijkstra.

- **Nós simples e duplos:** usados nas listas acima conforme a necessidade de iteração ou retrocesso.

Foram implementados três modos de semáforo:

- 1) **Ciclo Fixo:** transições com tempo estático.
- 2) **Tempo de Espera:** adapta o tempo de sinal conforme a quantidade de carros.
- 3) **Consumo:** alterna entre os dois modos anteriores, ativamente tempo de espera em horários de pico.

A. Modelagem do Sistema

O sistema é composto por duas entidades principais: **ruas** e **interseções**. Cada *rua* é capaz de calcular o **tempo de travessia**, o **consumo por travessia** e a **capacidade de veículos** com base em seu comprimento. Para isso, foi adotado o comprimento médio de um carro como 4 metros, permitindo estimar a quantidade máxima de veículos suportados proporcionalmente ao tamanho da via.

A classe *Veiculo*, ao atravessar uma sequência de ruas, acumula o tempo de viagem e o consumo de combustível baseado nas características de cada rua. Esses dados são armazenados internamente para análise posterior do percurso completo.

As *interseções* foram modeladas com dois semáforos: um para o fluxo Norte-Sul e outro para Leste-Oeste. Cada semáforo possui um atributo booleano *isVermelho*, indicando se o sinal está fechado. Como os veículos têm acesso às ruas diretamente, o sistema verifica qual direção está sendo tomada para consultar o semáforo correspondente. Se, por exemplo, o veículo entrar em uma rua horizontal, o estado do semáforo Leste-Oeste é avaliado para autorizar ou bloquear o movimento.

A movimentação dos veículos ocorre a partir de caminhos gerados aleatoriamente pela classe *Roteador*, que simula rotas urbanas utilizando algoritmos de busca. Durante a travessia, o veículo avalia o semáforo da próxima interseção e avança apenas quando o sinal está verde, respeitando o ciclo semafórico definido na lista circular.

VIII. RESULTADOS E LIMITAÇÕES

Durante o desenvolvimento, enfrentamos dificuldades com a manipulação de arquivos JSON em Java para gerar o grafo original. Por isso, criamos um novo grafo fixo com 4 interseções e 8 ruas (grafo bidirecional em matriz 2x2).

Apesar disso, persistiram falhas na aplicação do algoritmo de Dijkstra, que impossibilitaram o uso completo dos semáforos na simulação.

IX. CONCLUSÃO

O uso de listas circulares encadeadas mostrou-se eficaz para modelar sistemas semafóricos cíclicos. O sistema de Grafos deve ser recodificado. Após isso torna-se mais fácil a implementação dos outros sistemas.

REFERÊNCIAS

- [1] A. M. L. Mello, P. A. M. Melo, and F. R. Soares, “Avaliação de Desempenho de Tráfego Urbano Usando Simulação – Estudo de Caso em Maceió,” *Universidade Federal de Alagoas*, 2018. <https://www.repositorio.ufal.br/handle/123456789/10016>
- [2] C. M. Barros and D. G. de Faria, “Uso de Simuladores de Direção Aplicado ao Projeto de Segurança Viária,” *Revista Transporte e Desenvolvimento*, 2014. <https://www.researchgate.net/publication/265851573>
- [3] M. L. M. Santos, “Utilização da simulação computacional para análise do fluxo de veículos em um cruzamento com interseções semaforizadas,” *UFOP*, 2017. <https://monografias.ufop.br/handle/35400000/661>
- [4] E. de Almeida, “Simulação do Fluxo de Veículos de uma Rotatória Localizada na Cidade de Manaus,” 2019. <https://www.academia.edu/39269092>
- [5] G. D. Junior, R. Frozza, and R. F. Molz, “Simulação de controle adaptativo de tráfego urbano por meio de sistema multiagentes,” *Revista Brasileira de Computação Aplicada*, 2015. <https://seer.upf.br/index.php/rbca/article/view/4697>
- [6] T. H. Cormen et al., *Algoritmos: Teoria e Prática*, 3ª ed., Elsevier, 2012.
- [7] J. P. Gomes and A. L. Silva, “Modelagem de Sinais Semaforicos com Listas Circulares Encadeadas,” *Revista de Engenharia e Tecnologia Aplicada*, vol. 3, pp. 45–52, 2015. <https://www.revistas.unijui.edu.br/index.php/reta/article/view/4048>
- [8] V. R. Araújo et al., “Simulação de controle de tráfego com estruturas de dados dinâmicas em Java,” *Anais do Congresso Brasileiro de Computação*, 2019. <https://www.researchgate.net/publication/335445372>
- [9] P. A. Lopes et al., “Ensino de estruturas de dados com foco em aplicações práticas: uma abordagem baseada em simulações visuais,” *Anais do Workshop de Informática na Escola*, 2017. <https://sol.sbc.org.br/index.php/wie/article/view/3195>